

Digital signatures

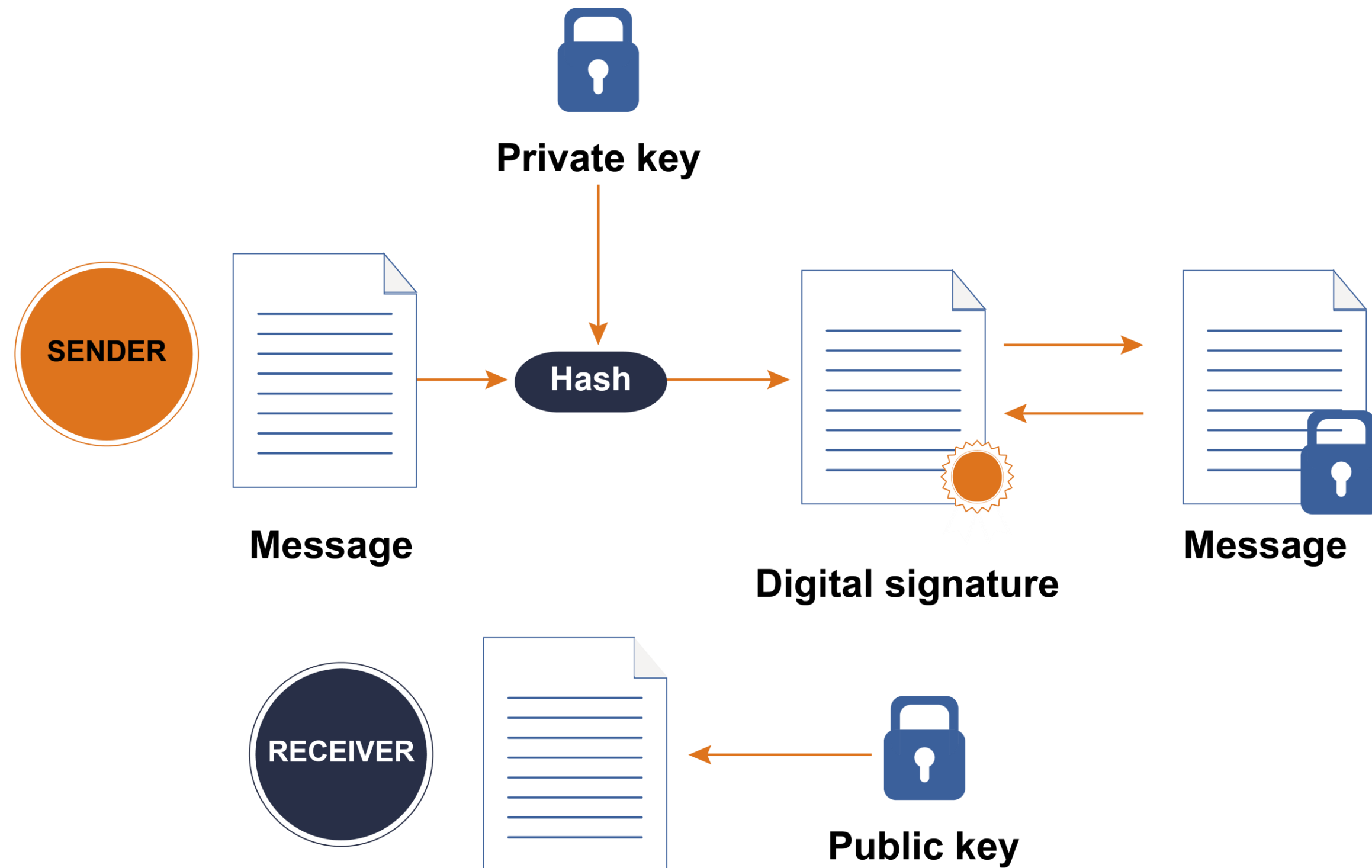
Learning objectives

- **Explain how digital signatures work**
- **Explore signing, verification and key roles**
- **Apply to real-world applications and example code**

What is a digital signature?

- **Mathematical method for authenticity and integrity**
- **Works like a handwritten signature in digital form**
- **Proves origin and prevents tampering**

Visual process overview



Key concepts



PRIVATE KEY
FOR SIGNING



PUBLIC KEY
FOR VERIFICATION



HASH FUNCTION
FOR MESSAGE
SUMMARY



CERTIFICATION
LINKS PUBLIC KEY
TO IDENTITY

Use case: email signing

- **Confirms sender and message integrity**
- **Protects against spoofing and fraud**
- **Used in secure and enterprise email systems**

Use case: software signing

- **Validates software source and integrity**
- **Prevents tampered updates and malware**
- **Trusted in operating system (OS) and app distributions**

Use case: legal documents

- **Proves document authorship and time**
- **Recognised under digital signature laws**
- **Used in e-signatures and banking and digital contracts in procurement systems**

Signature algorithms

**Rivest-Shamir-
Aldeman**

(RSA)

**Based on the factorisation of
large numbers**

**Supports signing and
encryption**

**Digital Signature
Algorithm**

(DSA)

**US government-
backed algorithm**

**Used in secure
applications**

**Elliptic Curve Digital
Signature Algorithm**

(ECDSA)

**More efficient and
secure with shorter
keys**

Example in Python

- **Use hashlib and Crypto modules**
- **Generate a key pair and sign a message**
- **Verify using a public key and SHA256**

```
import hashlib
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

key = RSA.generate(2048)
message = b"Secure message"

h = SHA256.new(message)
signature = pkcs1_15.new(key).sign(h)

# Verification
try:
    pkcs1_15.new(key.publickey()).verify(h, signature)
    print("Signature is valid")
except (ValueError, TypeError):
    print("Signature is invalid")
```

Strengths and limitations

- **Strong against forgery and tampering**
- **Depends on private key security**
- **Trust requires certificate authorities**

Summary

- **Digital signatures confirm identity and data**
- **Used in secure messaging and software delivery**

Next

- **Common cryptographic applications**